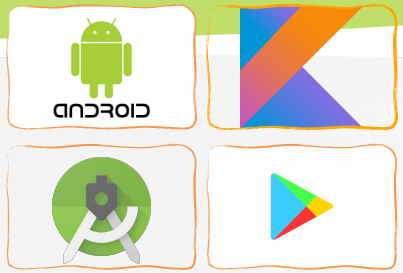
Software Development for Mobile Devices

Nguyen Le Hoang Dung
nlhdung@fit.hcmus.edu.vn

Content



- User Interface
- WYSIWYG GUI editor
- Layouts
- Basic widgets
  - textview, image, button, checkbox, radioButton
- Practice 1,2

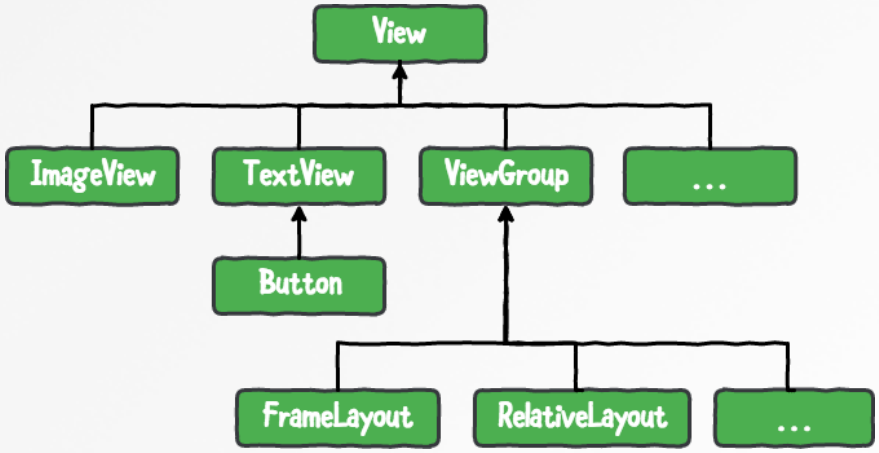
Graphical User Interfaces

The **VIEW Class**: Android's most basic component from which users interfaces can be created. It acts as a container of displayable elements.

» A **View** occupies a rectangular area on the screen and is responsible for drawing and event handling.

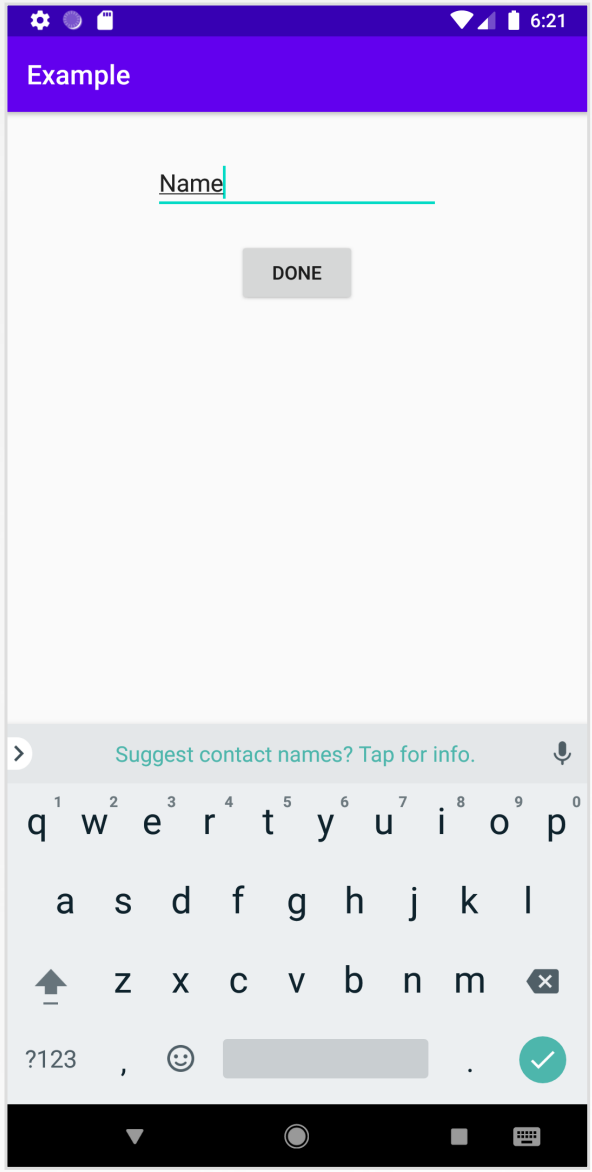
» **Widgets** are subclasses of View. They are used to create interactive UI components such as buttons, checkboxes, labels, text fields, etc…

» **Layouts** are invisible structured containers used for holding other Views and nested layouts



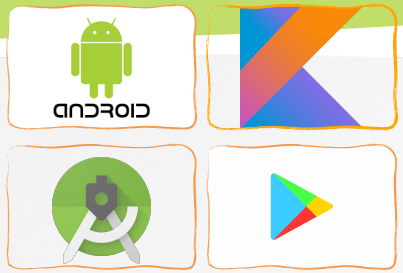
Graphical User Interfaces (cont.)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/constraintLayout"
    tools:context=".MainActivity">

    <EditText
        android:id="@+id/editText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="30dp"
        android:ems="10"
        android:inputType="textPersonName"
        android:text="Name"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:text="Done"
        app:layout_constraintEnd_toEndOf="@+id/editText"
        app:layout_constraintStart_toStartOf="@+id/editText"
        app:layout_constraintTop_toBottomOf="@+id/editText" />
</androidx.constraintlayout.widget.ConstraintLayout>
```



Actual UI displayed by the app

Graphical User Interfaces (cont.)



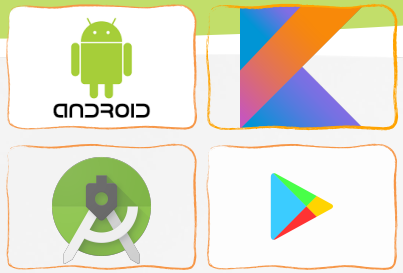
Nesting XML Layouts

» An Android's XML view file consists of a layout design holding a hierarchical arrangement of its contained elements.

» The inner elements could be basic widgets or user-defined nested layouts holding their own viewgroups.

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal" >

    Widgets and other nested layouts

</LinearLayout>
```

Graphical User Interfaces (cont.)



Setting Views to Work

» Dealing with widgets & layouts typically involves the following operations

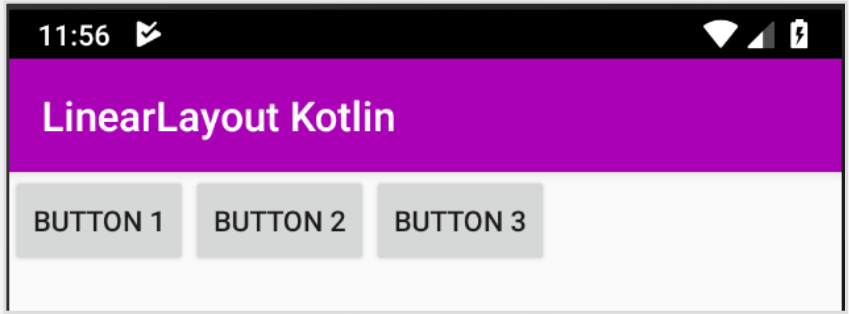
- Set properties: you can set the background color, text, font, alignment, size, padding, marging, etc (for TextView)
- Set up listeners: For example, an image could be programmed to respond to various events such as: click, long-tap, mouse-over, etc.
- Set focus: To set focus on a specific view, you call the method .requestFocus() or use XML tag <requestFocus />
- Set visibility: You can hide or show views using setVisibility(...).

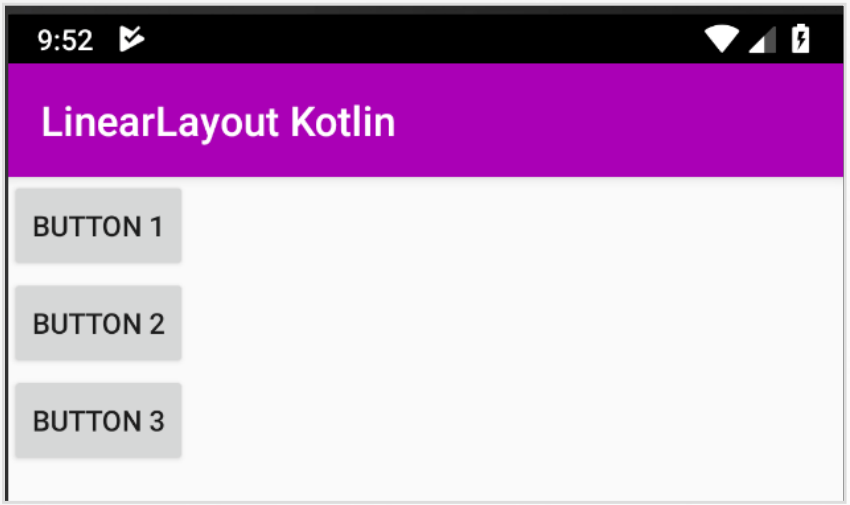
Graphical User Interfaces (cont.)

Common Android LAYOUTS

Dealing with widgets & layouts typically involves the following operations

- **Linear Layout:** places its inner views either in horizontal or vertical disposition.



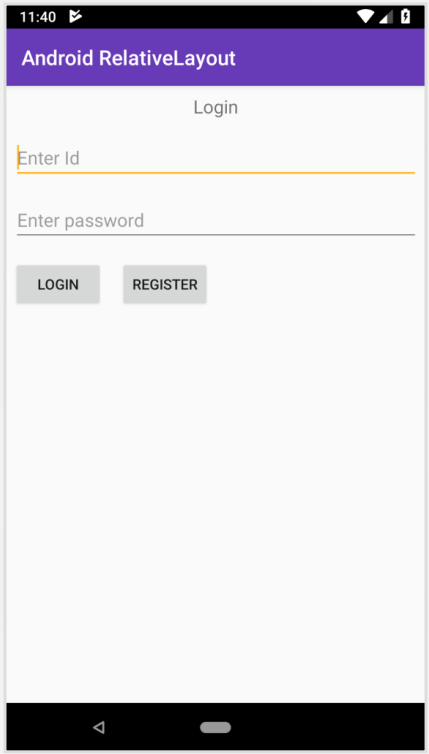


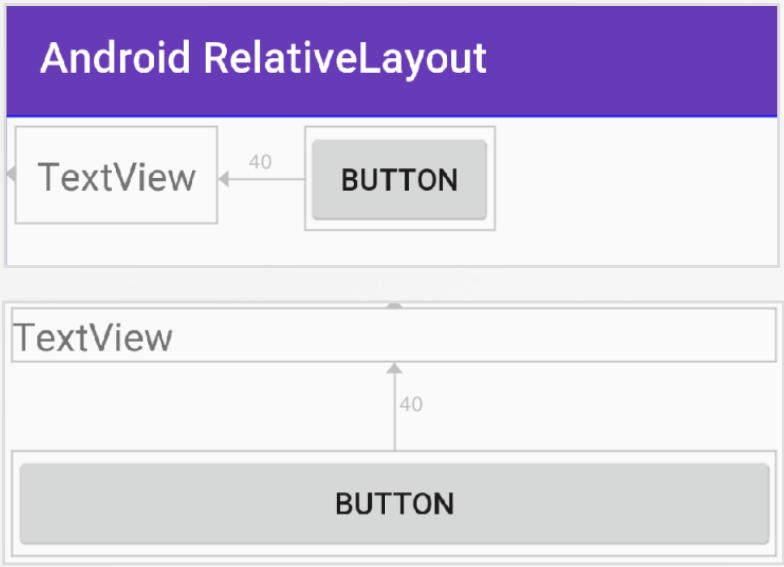
https://developer.android.com/guide/topics/ui/declaring-layout

Graphical User Interfaces (cont.)

Common Android LAYOUTS

- **Relative Layout:** a ViewGroup that allows you to position elements relative to each other

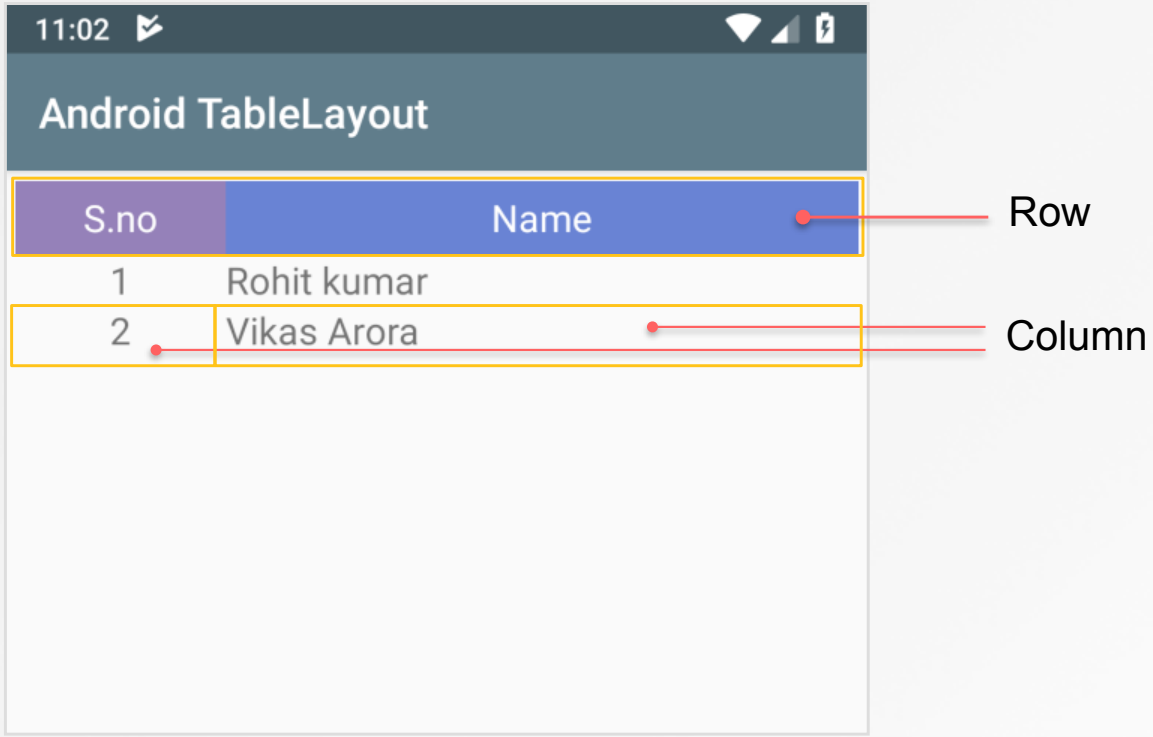




Graphical User Interfaces (cont.)

Common Android LAYOUTS

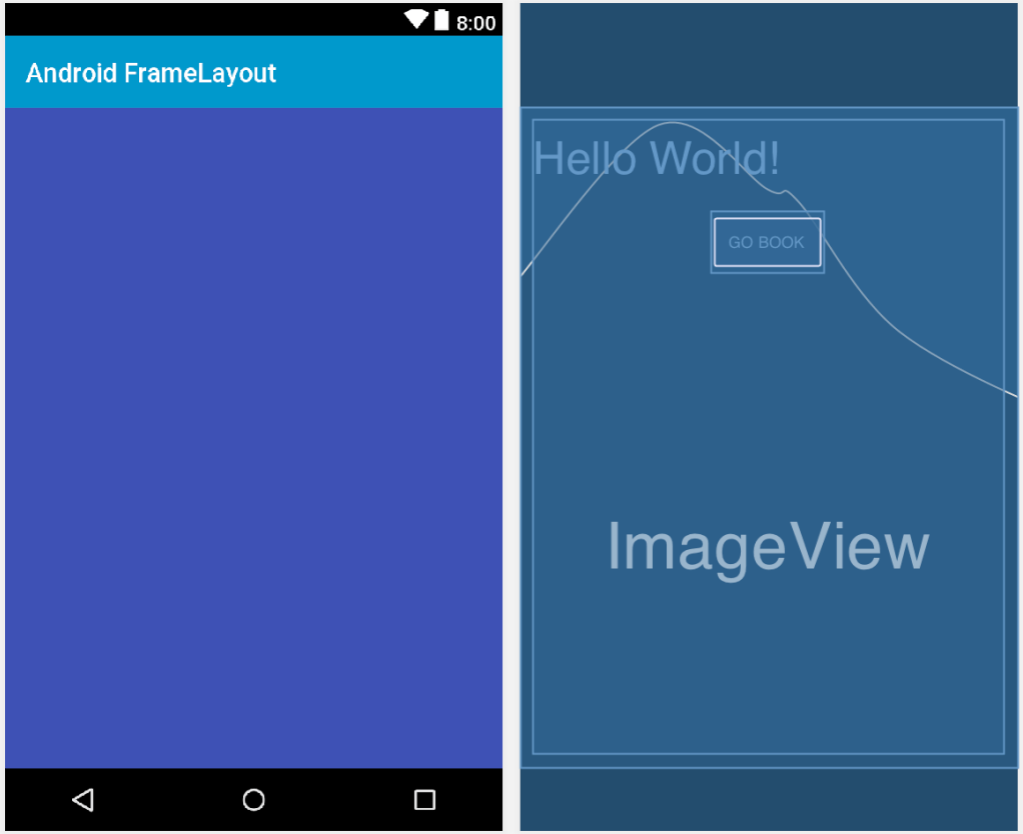
- **Table Layout:** a ViewGroup that places elements using a row & column disposition

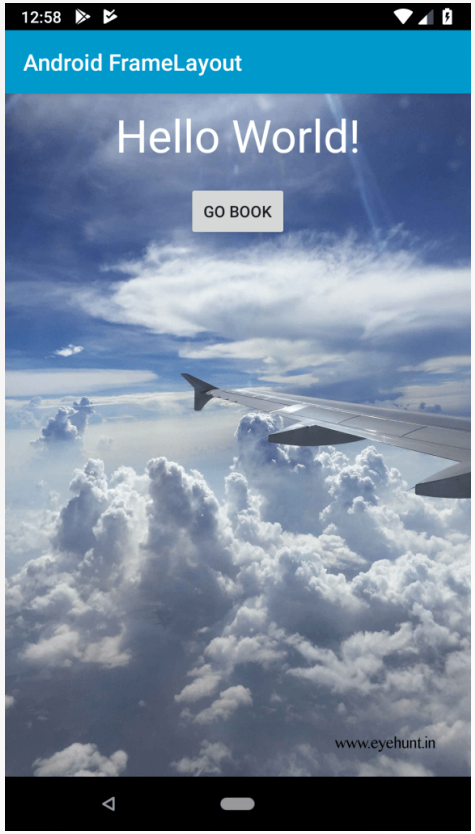


Graphical User Interfaces (cont.)

Common Android LAYOUTS

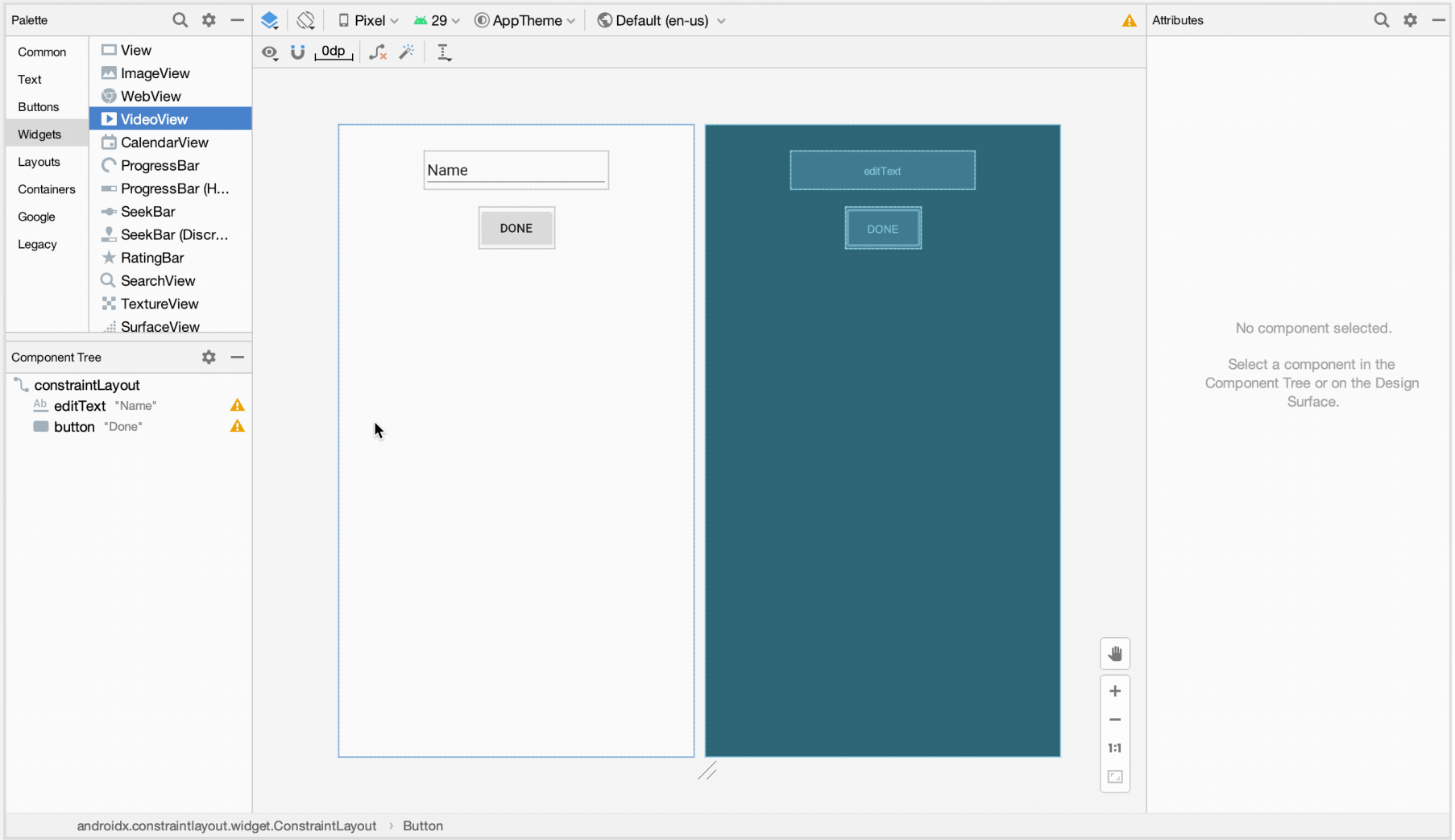
- **Frame Layout:** views on the top of other views.



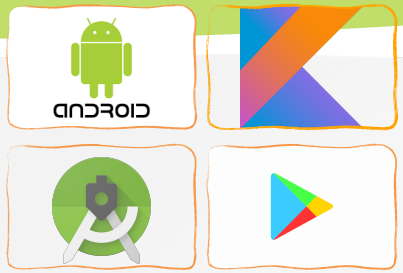


Graphical User Interfaces (cont.)

GUI Editing: WYSIWYG Version with other Controls



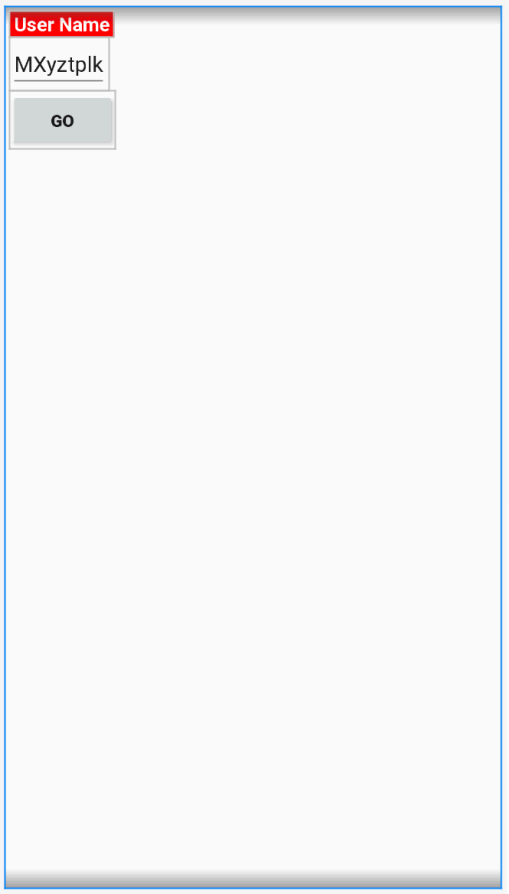
Linear Layout



- Setting Attributes
  - orientation: vertical, horizontal
  - fill model: match_parent, wrap_contents
  - gravity: top, bottom, center…
  - padding: some pixels
  - margin: some pixels
  - weight: 0,1,2, … n

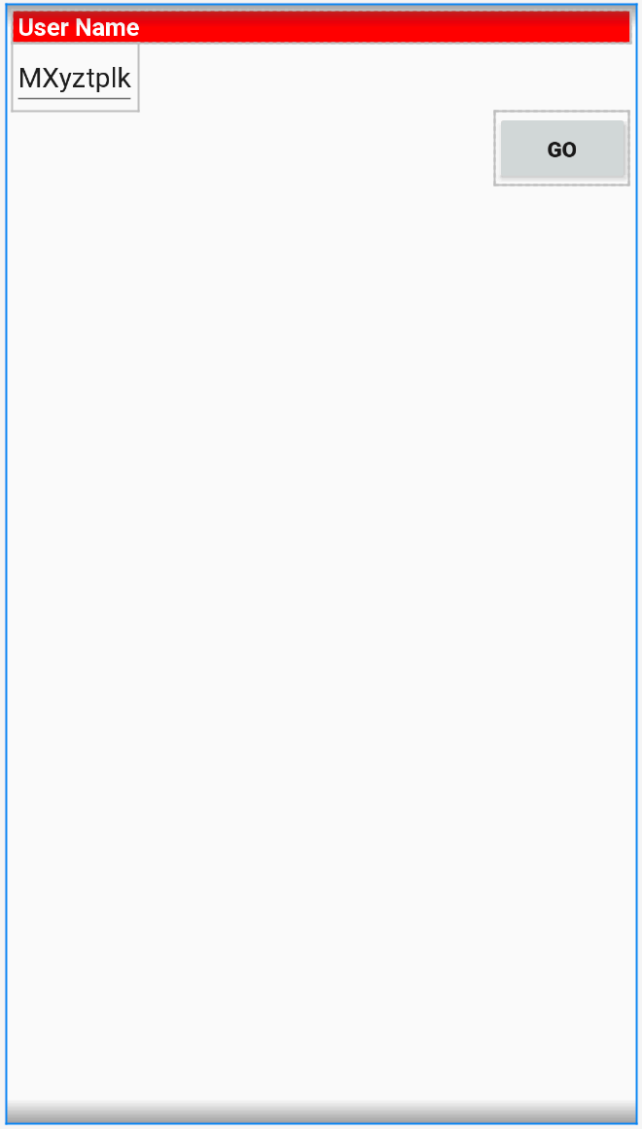
Linear Layout (cont.)

```
<?xml version="1.0" encoding="utf-8" ?>
<LinearLayout xmlns:android="http://
schemas.android.com/apk/res/android"
    android:id="@+id/constraintLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal"
    android:padding="4dp">
    <TextView android:id="@+id/labelUserName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:background="#ffff0000"
        android:text=" User Name "
        android:textColor="#ffffffff"
        android:textSize="16sp"
        android:textStyle="bold" />
    <EditText
        android:id="@+id/ediName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="MXyztplk"
        android:textSize="18sp" />
    <Button
        android:id="@+id/btnGo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Go"
        android:textStyle="bold" />
</LinearLayout>
```



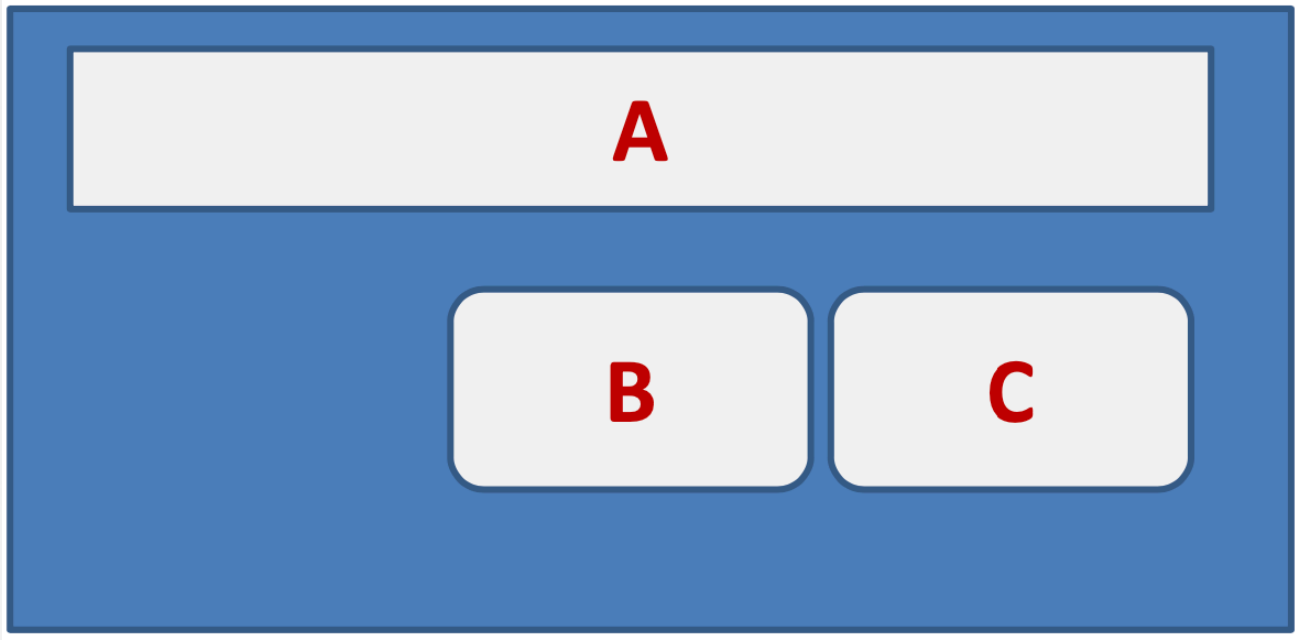
Linear Layout (cont.)

```
<?xml version="1.0" encoding="utf-8" ?>
<LinearLayout xmlns:android="http://
schemas.android.com/apk/res/android"
    android:id="@+id/constraintLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="4dp">
    <TextView android:id="@+id/labelUserName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="#ffff0000"
        android:text=" User Name "
        android:textColor="#ffffffff"
        android:textSize="16sp"
        android:textStyle="bold" />
    <EditText
        android:id="@+id/ediName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="MXyztplk"
        android:textSize="18sp" />
    <Button
        android:id="@+id/btnGo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="right"
        android:text="Go"
        android:textStyle="bold" />
</LinearLayout>
```



Relative Layout

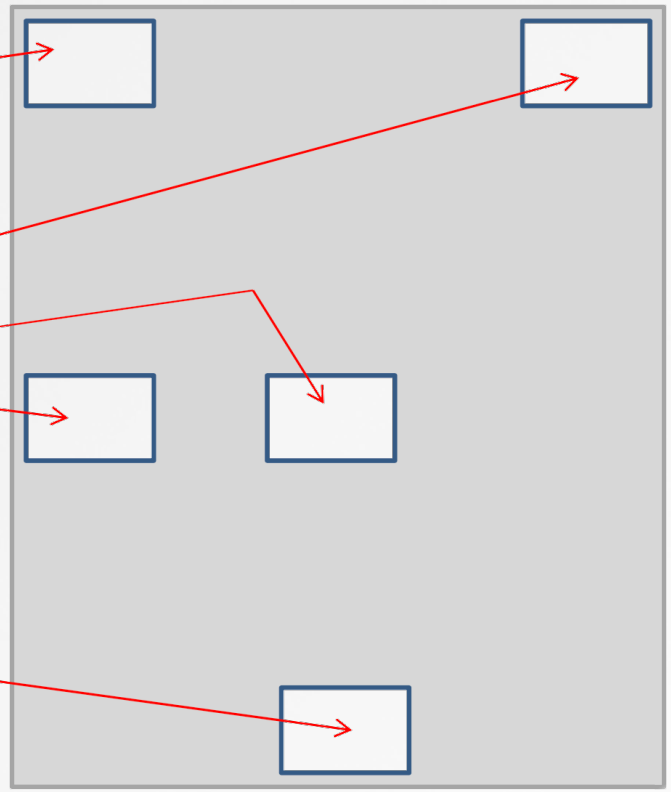
A is by the parent's top
C is below A, to its right
B is below A, to the left of C



Relative Layout (cont.)

Referring to the container

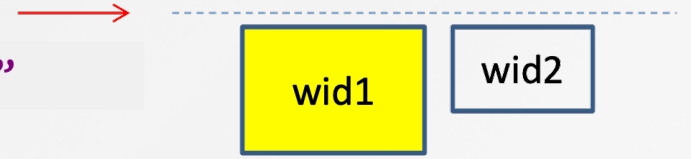
```
android:layout_alignParentTop
android:layout_alignParentBottom

android:layout_alignParentLeft
android:layout_alignParentRight

android:layout_centerInParent
android:layout_centerVertical
android:layout_centerHorizontal
```



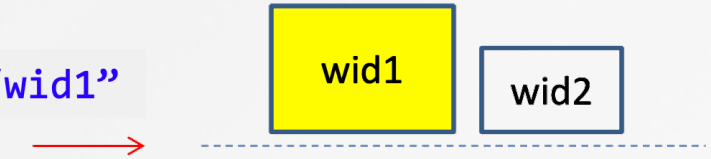
Relative Layout (cont.)

Referring to Other Widgets **android:id="@+id/yourID"**

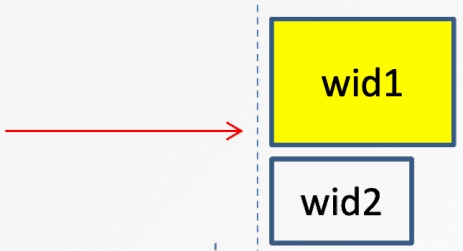
```
android:layout_alignTop="@+id/wid1"
```



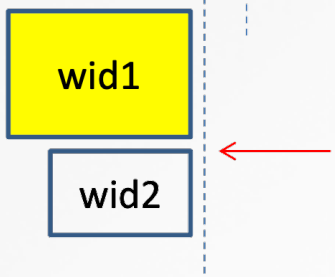

```
android:layout_alignBottom ="@+id/wid1"
```




```
android:layout_alignLeft="@+id/wid1"
```

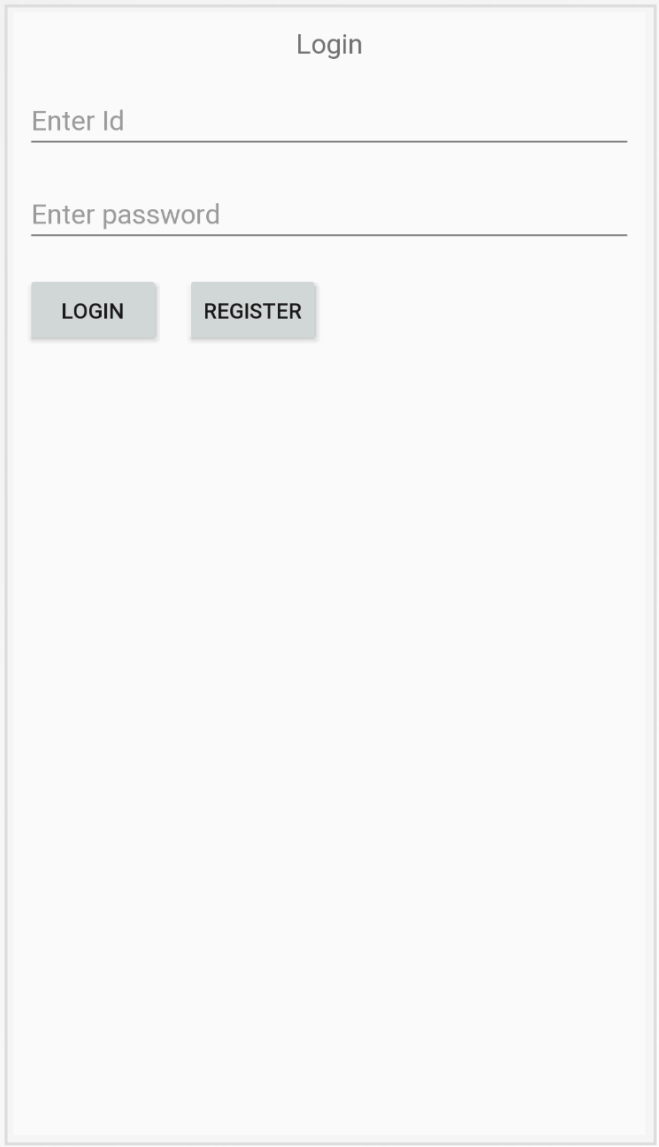



```
android:layout_alignRight="@+id/wid1"
```



Relative Layout (cont.)

```
<?xml version="1.0" encoding="utf-8" ?>
<RelativeLayout xmlns:android="http://
schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/tv_label"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentTop="true"
        android:layout_centerHorizontal="true"
        android:text="Login"
        android:textSize="18sp"
        android:layout_margin="8dp"/>

    <EditText
        android:id="@+id/et_id"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="8dp"
        android:layout_below="@+id/tv_label"
        android:ems="10"
        android:hint="Enter Id"
        android:inputType="textEmailAddress" />
```



Relative Layout (cont.)

```
<EditText
    android:id="@+id/et_pass"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_below="@+id/et_id"
    android:layout_margin="8dp"
    android:ems="10"
    android:hint="Enter password"
    android:inputType="textPassword" />

<Button
    android:id="@+id/btn_login"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@+id/et_pass"
    android:layout_margin="8dp"
    android:text="Login" />

<Button
    android:id="@+id/btn_reg"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@+id/et_pass"
    android:layout_margin="8dp"
    android:layout_toRightOf="@+id/btn_login"
    android:text="Register" />

</RelativeLayout>
```

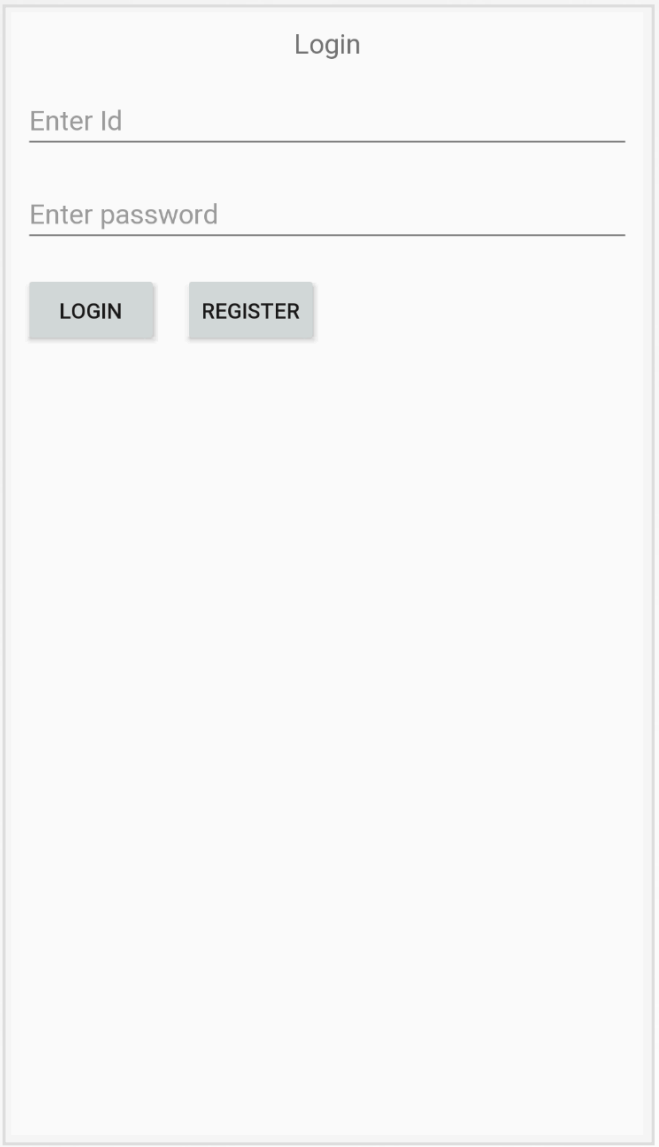
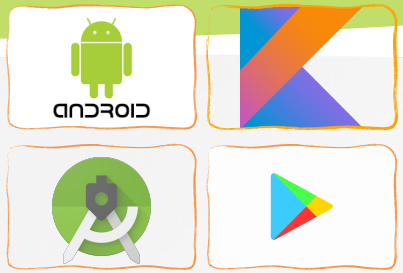


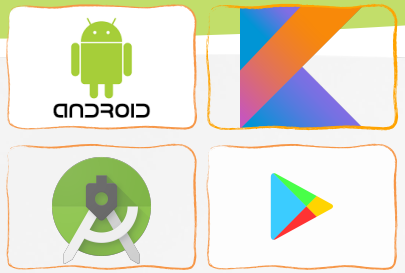
Table Layout



This layout uses a grid template to position your widgets.

- Like in a 2D matrix, cells in the grid are identified by **rows** and **columns**.
- Columns are flexible, they could shrink or stretch to accommodate their contents.
- The element **TableRow** is used to define a new row in which widgets can be allocated.
- The **number of columns** in a TableRow is determined by the **total of side-by-side widgets** placed on the row.

Table Layout (cont.)



```
<?xml version="1.0" encoding="utf-8" ?>
<TableLayout xmlns:android="http://
schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="5dp"
    android:stretchColumns="2"
    tools:context=".MainActivity">

    <TableRow
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <TextView
            android:layout_width="100dp"
            android:layout_height="35dp"
            android:layout_column="1"
            android:background="@color/colorAccent"
            android:gravity="center"
            android:text="S.no"
            android:textSize="18sp" />
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="35dp"
            android:layout_column="2"
            android:background="@color/colorAccent"
            android:gravity="center"
            android:text="Name"
            android:textSize="18sp" />
    </TableRow>
```

Col index for matching container

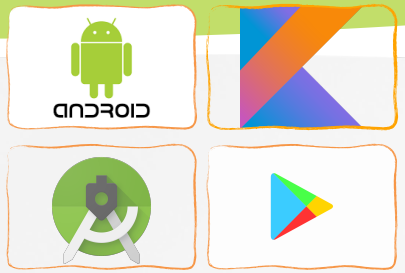
…="1,2" (equal)

```
    <TableRow>
        <TextView
            android:layout_width="100dp"
            android:layout_height="wrap_content"
            android:layout_column="1"
            android:gravity="center"
            android:text="1"
            android:textSize="18sp" />
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_column="2"
            android:text="Dung Nguyen"
            android:gravity="center"
            android:textSize="18sp" />
    </TableRow>
    <TableRow>
        …
    </TableRow>
     <TableRow>
        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_column="1"
            android:layout_span="2"
            android:background="#8BC34A"
            android:text="Summary"
            android:textSize="18sp" />
     </TableRow>
</TableLayout>
```

Skip a column

Occupy 2 cols

Table Layout (cont.)



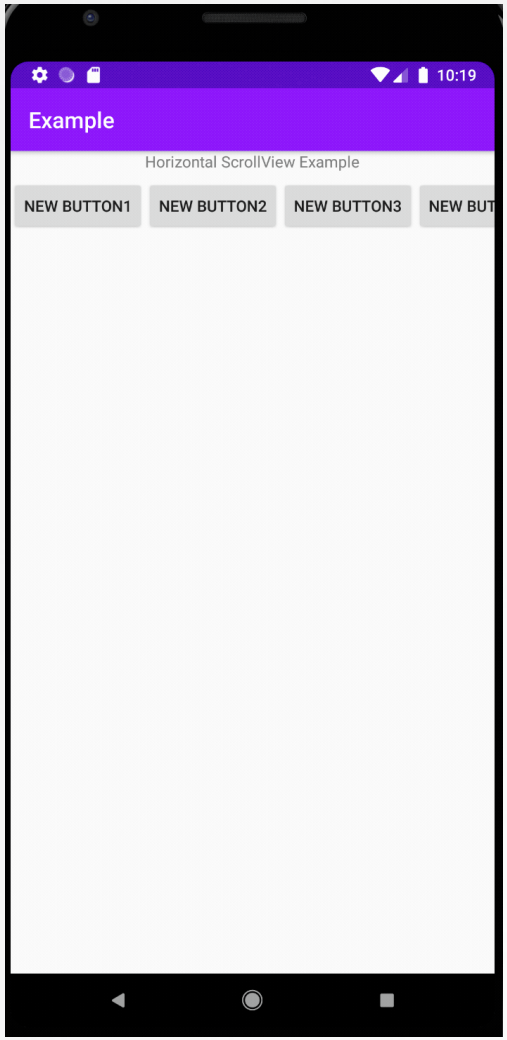
```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://
schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="5dp"
    android:stretchColumns="2"
    tools:context=".MainActivity">

    <TableRow
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <TextView
            android:layout_width="100dp"
            android:layout_height="35dp"
            android:layout_column="1"
            android:background="@color/colorAccent"
            android:gravity="center"
            android:text="S.no"
            android:textSize="18sp" />
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="35dp"
            android:layout_column="2"
            android:background="@color/colorAccent"
            android:gravity="center"
            android:text="Name"
```

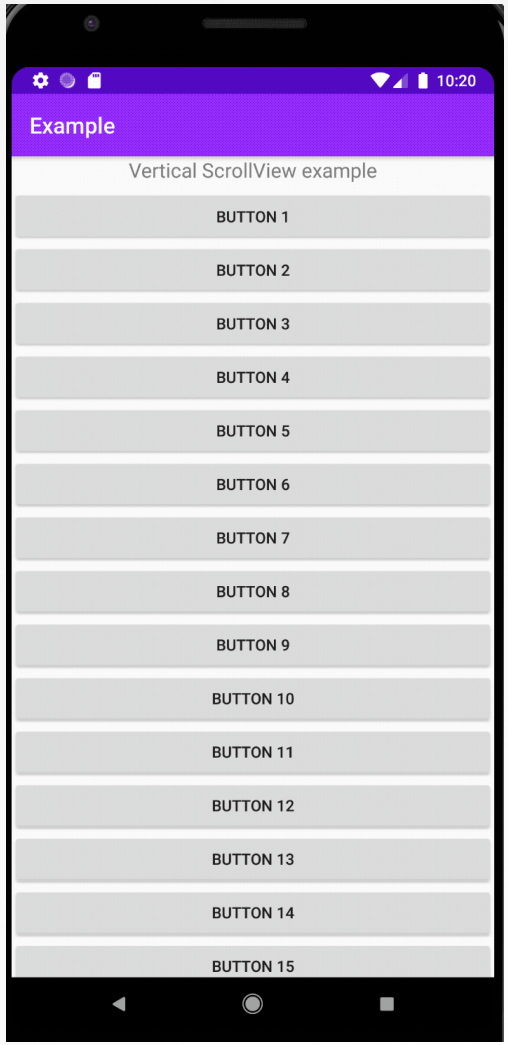
```
    <TableRow>
        <TextView
            android:layout_w
            android:layout_h
            android:layout_c
            android:gravity="
            android:text="1"
            android:textSize=
        <TextView
            android:layout_w
            android:layout_h
            android:layout_c
            android:text="Du
            android:gravity="
            android:textSize=
    </TableRow>
    <TableRow>
        ...
    </TableRow>
    <TableRow>
        <TextView
            android:layout_
            android:layout_
            android:layout
            android:layout
            android:backg
            android:text="
            android:textSize
    </TableRow>
</TableLayout>
```

S.no	Name
1	Dung Nguyen
2	Mister Xyztplk
Summary	

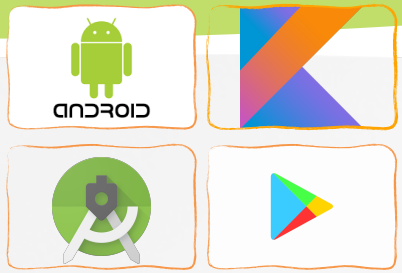
ScrollView Layout (Vertical & Horizontal)

- The ScrollView control is useful in situations in which we have more data to show than what a single screen could display.
  - It provides a vertical sliding (up/down) access to the data.
  - The HorizontalScrollView provides a similar left/right sliding mechanism)
  - Only a portion of the user's data can be seen at one time, however the rest is available for viewing.





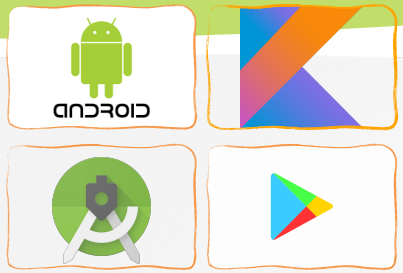
ScrollView Layout (Vertical & Horizontal)



```
<ScrollView android:layout_marginTop="30dp"
      android:layout_width="fill_parent"
      android:layout_height="wrap_content"
      android:id="@+id/scrollView">
      <LinearLayout
         android:layout_width="fill_parent"
         android:layout_height="fill_parent"
         android:orientation="vertical" >

         <Button
            android:layout_width="fill_parent"
            android:layout_height="wrap_content"
            android:text="Button 1" />
         <Button … />
         <Button … />
         <Button
            android:layout_width="fill_parent"
            android:layout_height="wrap_content"
            android:text="Button 20" />
      </LinearLayout>
   </ScrollView>
```

```
<HorizontalScrollView
   android:layout_width="match_parent"
   android:layout_height="60dp"
   android:id="@+id/horizontalScrollView">
   <LinearLayout
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:orientation="horizontal">
      <Button
         android:layout_width="wrap_content"
         android:layout_height="wrap_content"
         android:text="New Button1"
         android:id="@+id/button1" />
      <Button …  />
      <Button …  />
      <Button
         android:layout_width="wrap_content"
         android:layout_height="wrap_content"
         android:text="New Button8"
         android:id="@+id/button8"/>
   </LinearLayout>
</HorizontalScrollView>
```

Connecting Layouts to Java Code

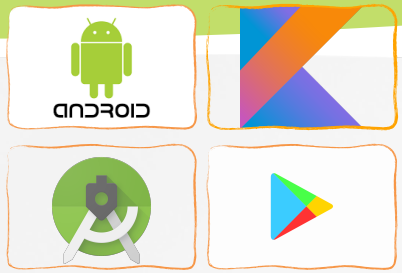


- Assume the UI in res/layout/activity_main.xml has been created. This layout could be called by an application using the statement
  - setContentView(R.layout.*activity_main*)
- Individual XML defined widgets, such as button1 is later associated to the application using the statement findViewByID(…)

```
val button1 = findViewById<Button>(R.id.button1)
button1.setOnClickListener {
    Log.d("TAG", "clicked!")
}
```

- Where ***R is a class automatically generated*** to keep track of resources available to the application.
  - R.id... is the collection of widgets defined in the XML layout

Android Context



- The context defines a logical workspace which allows access to
  - application-specific resources (style, theme, images…)
  - and classes,
  - as well as up-calls for application-level operations such as launching activities, broadcasting and receiving intents…
- When a widget is created, it is attached to a particular Context. By means of its affiliation to that environment, it then could access other members of the hierarchy on which it has been collocated.
  - For a simple 'one activity app' -say MainActivity- the method **getApplicationContext()** and the reference **MainActivity.this** return the same result.
- An application could have several activities. Therefore, for a multi-activity app we have 1 app context, and a context for each of its activities, each good for accessing what is available in that context.

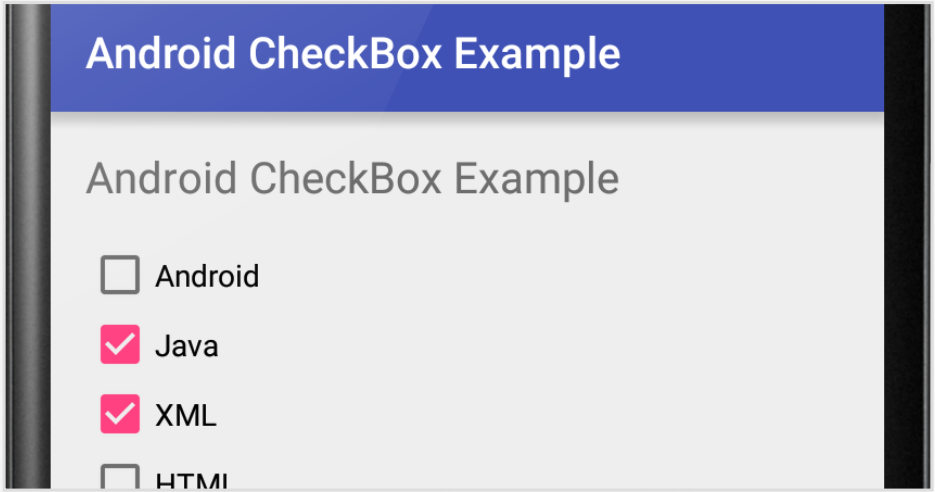
Basic widgets

- **Buttons**
- **TextViews**
- **ImageView** and **ImageButton** allow the embedding of images in your applications (gif, jpg, png, etc)
  - Pictures are stored in the ***res/drawable***
  - Set image programmatically

```
val imgageView = findViewById<ImageView>(R.id.imageView)
imageView.setImageResource(R.drawable.img1)

val imgBtn = findViewById<ImageButton>(R.id.imageButton)
imgBtn.setBackgroundResource(R.drawable.img1)
```

Basic widgets (cont.)

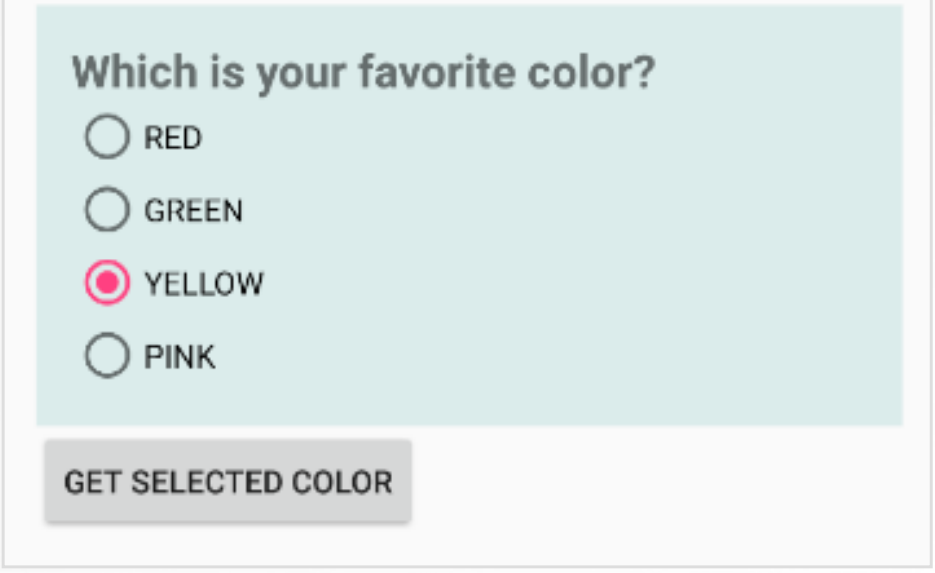
- **EditText**
- **CheckBoxes:** a special two-states button which can be either checked or unchecked.
  - ***isChecked*** property



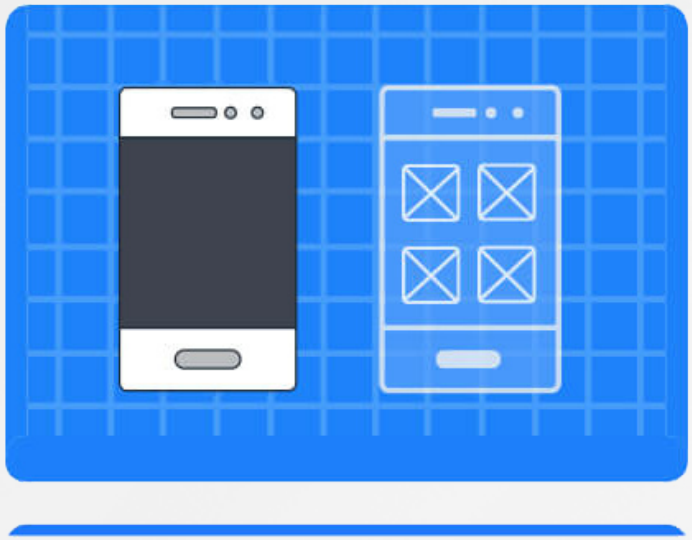
Basic widgets (cont.)

- **Radiobutton** (like a CheckBox) is a two-states button that can be either checked or unchecked.
  - Logically related radio buttons are normally put together in a **RadioGroup** container.
  - The container forces the enclosed radio buttons to behave as mutually exclusive selectors. That is, the checking of one radio button unchecks all the others.

```
val radioGroup =
    findViewById<RadioGroup>(R.id.radioGroup)
when(radioGroup.checkedRadioButtonId) {
    R.id.radioButton -> {
        println("Radio button 1 is selcted")
        println("Radio button 1 is selcted (duplicated)")
    }
    R.id.radioButton2 ->
        println("Radio button 2 is selcted")
    else -> println("Something else...")
}
```

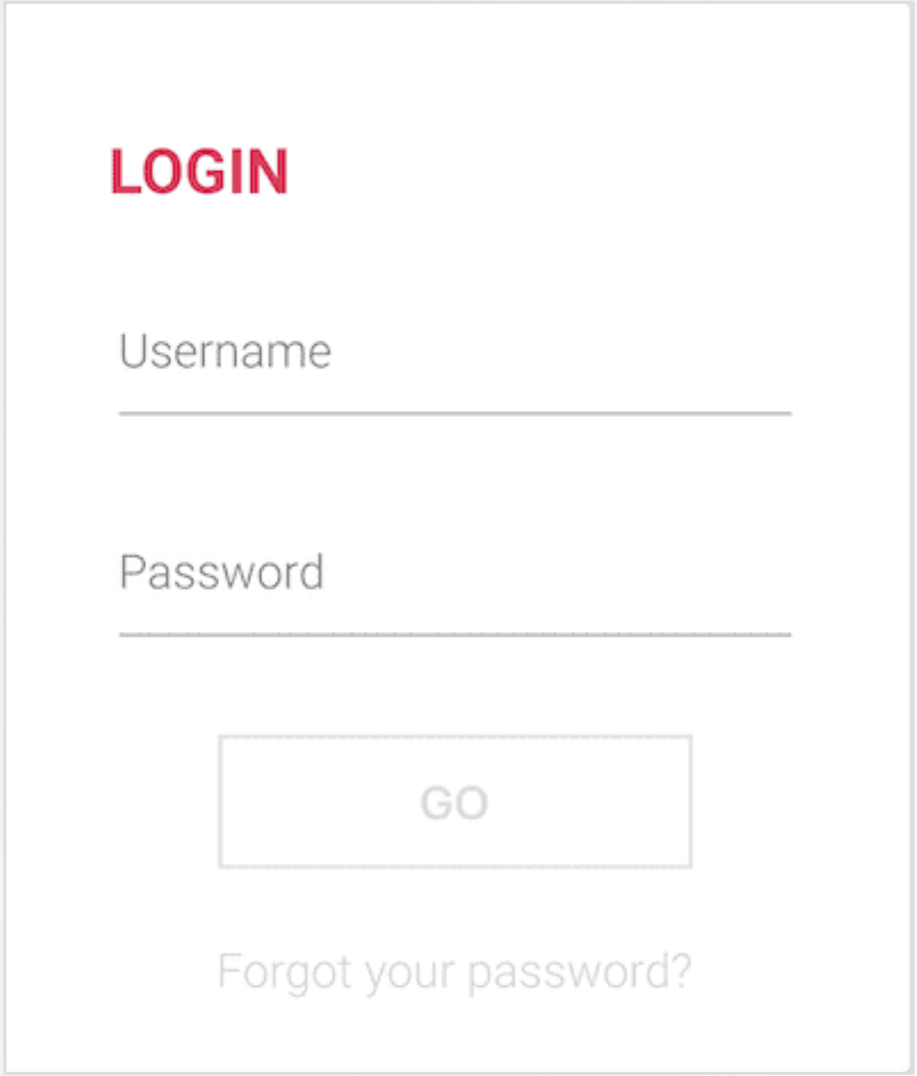


Demo



Kotlin

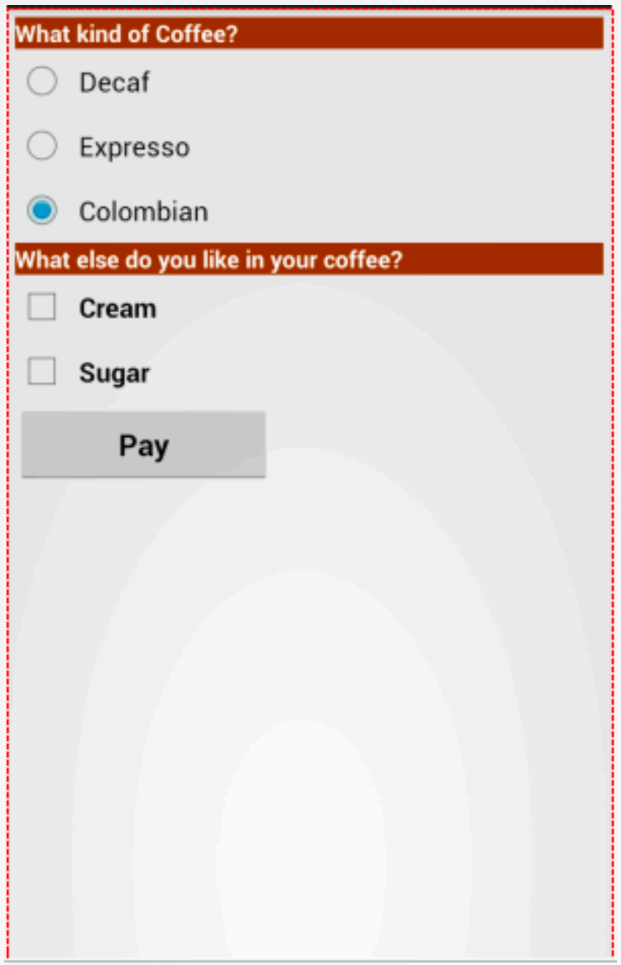
Simple Login Screen



Practice 1

- Implement the following project using these widgets:
  - RadioButton
  - Checkbox
  - Button
  - Textview
  - Toast

```
Toast.makeText(this, "[your text]" ,
        Toast.LENGTH_SHORT).show()
```



Practice 2.1

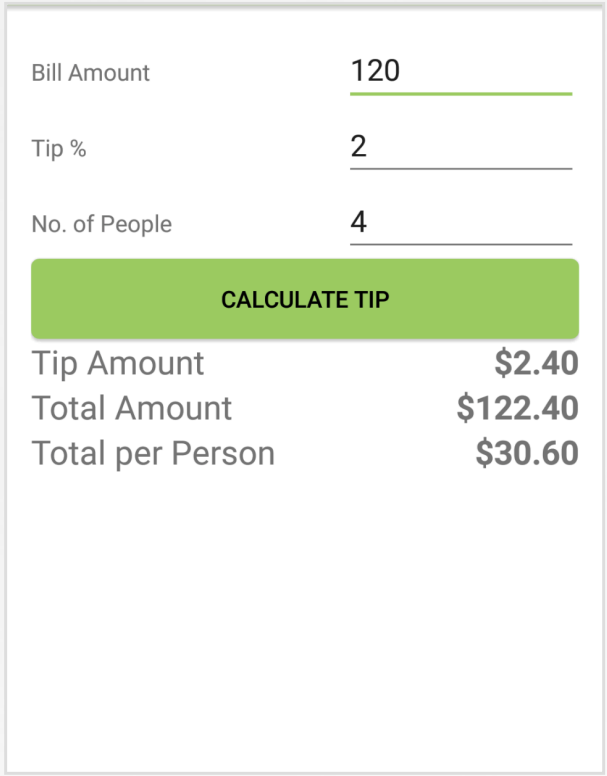
- Implement any/all of the following projects using simple UI controls (edittext, textview, button, imageview, imagebutton)

Currency Exchange calculator

Practice 2.2

- Implement any/all of the following projects using simple UI controls (edittext, textview, button, imageview, imagebutton)

Tip Calculator



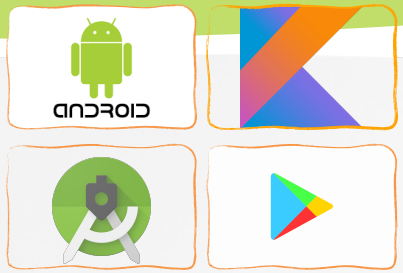
Practice 2.2

- Implement any/all of the following projects using simple UI controls (edittext, textview, button, imageview, imagebutton)

Simple Flashlight with background colors

Q&A

Reference



- https://kotlinlang.org/docs/reference/
- Peter Späth, Learn Kotlin for Android Development, 2019
- Ted Hagos, Learn Android Studio 3 with Kotlin, 2018
- Peter Späth, Pro Android with Kotlin, 2019
- Milos Vasic, Mastering Android Development with Kotlin, 2017
- Victor Matos, Mobile Application Development course, Cleveland State University
- https://developer.android.com/guide/
- https://developer.android.com/guide/topics/ui/declaring-layout